PROJECT REPORT FOR DESIGN AND SIMULATION OF A SELF-BALANCING BOT

Acknowledgements:

A great amount of dedicated effort has been put into the development of this project however it would not have been possible without the guidance and support of a few people.

I would first like to heartily thank Professor D. Ezhilarasi, from the Department of Instrumentation and Control Engineering, National Institute of Technology, Tiruchirappalli, for her guidance and support. I am also thankful to the staff for extending their help when it was most needed.

Aim:

The main aim of this project was to model a full-fledged self-balancing bot and then to use an appropriate simulation software to balance the bot within a given time frame with the help of a suitable control algorithm.

Methodology:

Firstly, a model of the self-balancing bot had to be created using a suitable CAD software. I choose SolidWorks as a platform to model and assemble the different components of the bot.

Some parts were modelled directly on SolidWorks. The remaining parts which include the electronic components, batteries and motors were imported from Grab cad, a library for CAD models.

I started by designing the chassis for the bot where most of the components would be kept. This was followed by designing the middle frame and the head of the bot, along with the supports that would be required to hold the different components in place. Finally, the wheels of suitable radius were designed separately. The outer portion of the wheels were made of a rubber grip to provide better traction.

Once this was done, I then decided to incorporate the following electronic components in the CAD model:

a) Arduino UNO microcontroller, which could be used for implementation of the PID control algorithm.
b) IMU sensor, which can be used to measure the tilt angles that can then be fed for processing to the microcontroller via the I2C interface.
c) Motor-driver to amplify and control the speed and direction of current that is being fed to the motor from the microcontroller.
d) Battery: Used as the main power source for the system, and is connected tot the motor driver.
e) Bluetooth module: This could be used to control the bot wirelessly.
f) N-20 motors: DC motors that are used to drive the wheels of the robot.

All of the above-mentioned components were imported into SolidWorks and were then incorporated into the assembly, with the help of suitable mates in SolidWorks, like the concentric mate, the co-incident mate and the advanced distance mate (just to name a few).
Once all the parts were assembled together a few important physical parameters were calculated using the mass properties tool in SolidWorks. These included the mass, the center of mass position and the moment of inertia about the axis of rotation.

a) Mass = 2725.75 grams
b) Centre of mass: (millimetres)
   X = -288.55
   Y = -88.65
   Z = 1.78
c) Ixx = 79225670.88 (moment of inertia about the axis of the wheels, which is basically our axis of rotation if we compare this system with that of an inverted pendulum)

Given below are the different views of the bot after the modelling was completed on SolidWorks:

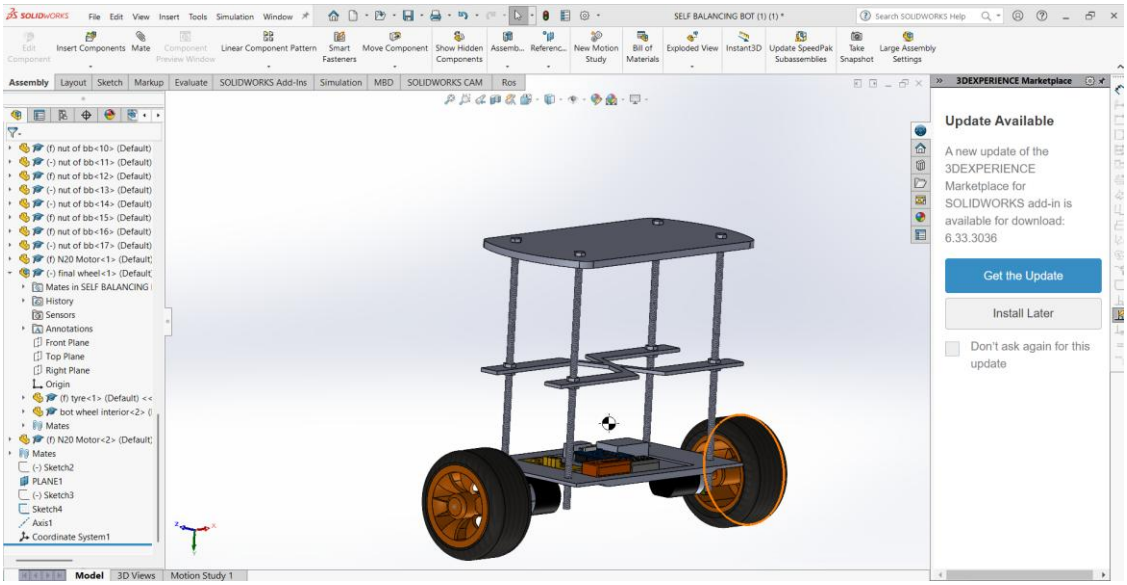


Fig.1 Isometric view

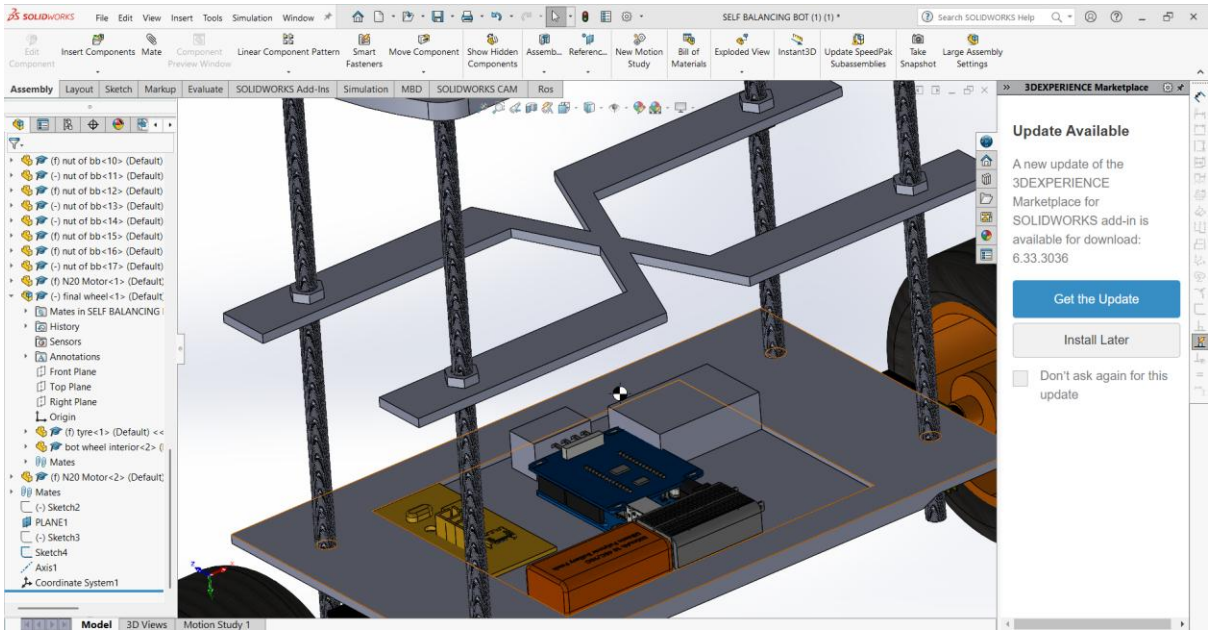


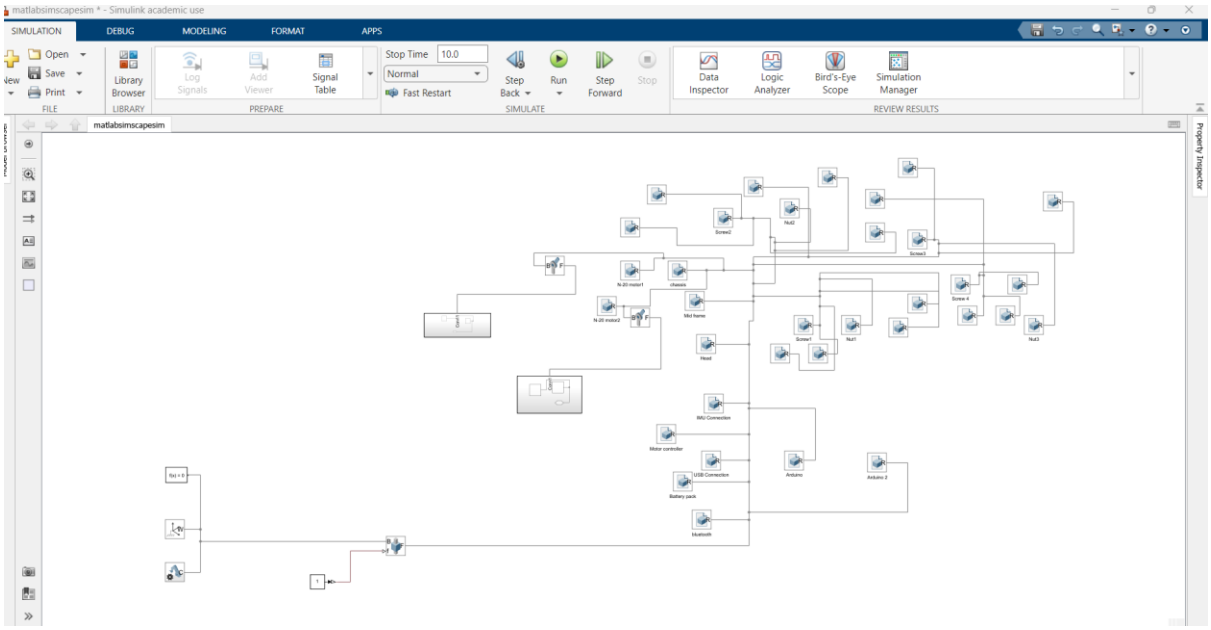
Fig.2 Zoomed in view of electronic components

After the modelling phase I moved on to the implementation of the control algorithms to balance the bot under a specified perturbation. This was done using Simulink and simscape multibody tool.

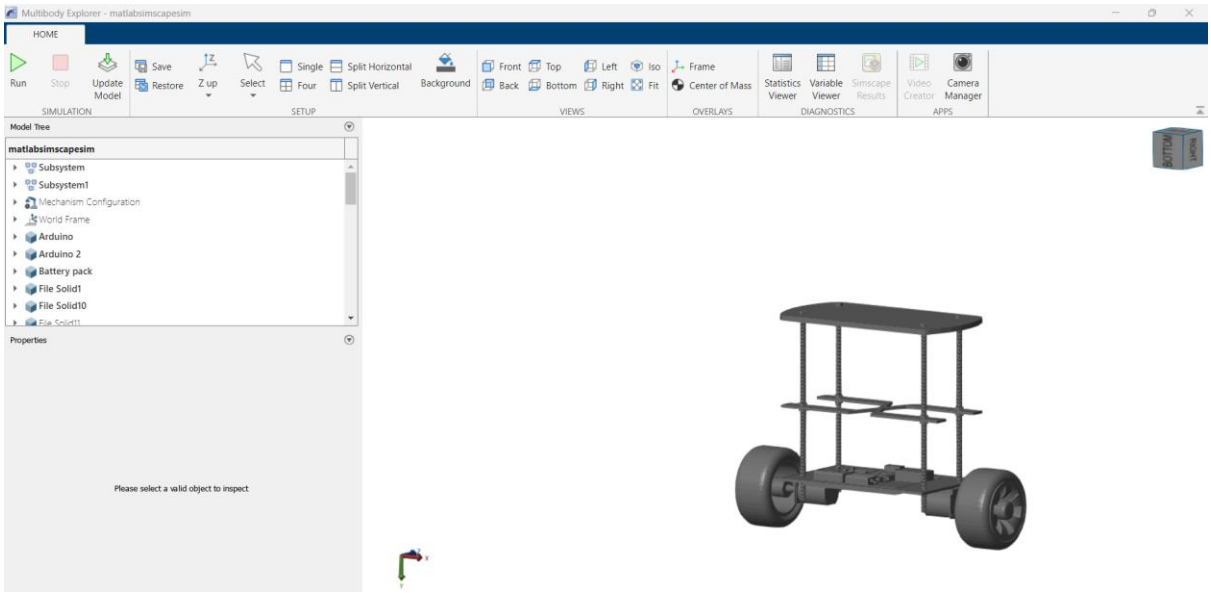
In order to import a CAD file into MATLAB, we need to use the Simscape multibody tool. The model was first converted into a folder consisting of the

.STL file of all the parts. Then, using the file solid block each part of the model was imported into the simscape Graphics-user interface workspace.

All the parts were then assembled by connecting all the file solid blocks suitably to get the desired model.



Each block in the above diagram represents a part or component of the final model of the bot. All of them are connected together to get the final bot in the Graphics interface of simscape as shown below:



Next, we need to develop a control algorithm on Simulink in order to control the angle of tilt of the balancing bot under major perturbations.

The following block diagram shows the broad idea involved behind developing a system to perform the given task:

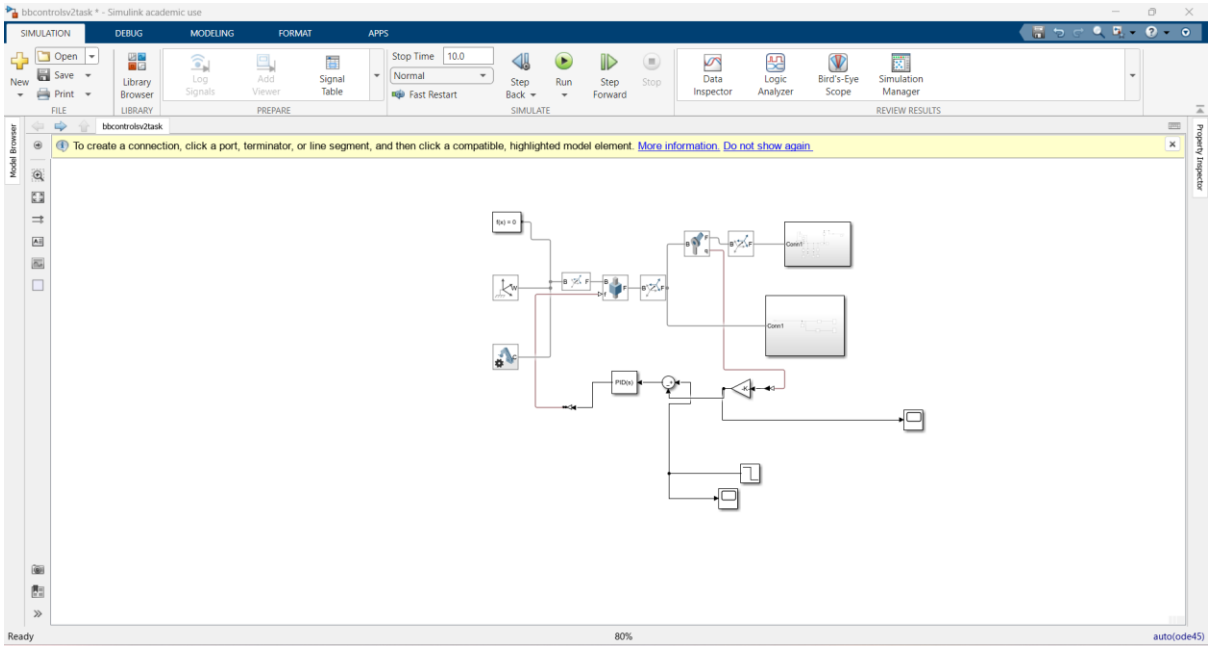


Fig 3. Simscape Block diagram of the closed loop control system

The following above system uses a PID controller block in order to balance the system.

First, I inserted the world frame, mechanism configuration and solver block in order to start with the system.

Simultaneously I also introduced the subsystems Conn1 and Conn2 which were connected together via a revolute joint block.

The revolute joint block is very crucial at this stage as it not only serves as a joint that enables rotation between the chassis and the axis but also helps in sensing the angle of tilt which is the output.

The combined system of conn1 and conn2 along with the revolute joints is then connected to the main world frame block via prismatic joints. This gives the bot an additional translational degree of freedom with respect to the world frame.

The output from the revolute joint block is then fed to the negative side of the PID block via the gain block that converts the angle into degrees.

The positive side of the PID block is connected to the output of the gain block that is connected to the revolute joint. At the start of our simulation the angle is 0 degrees.

Then we insert a step block that has an initial value of 10 degrees and a final value of 1 degree with a step time of 0.01 seconds.

The reason why we insert a step block is because the system is initially at equilibrium at T=0 seconds. The angle is also 0 degree. Then when we connect the step block with initial value of 10 degrees it acts like a sudden disturbance that is provided to the system. In 0.01 seconds the value of the step block changes to 1 degree which is the actual setpoint value that the controller has to achieve. (**Note: 10 degrees is not the setpoint value, it is just the initial value that is provided in order to give the required external disturbance to the system, the setpoint is 1 degree)**



Fig 4. Scope block that shows the initial drop from 10 degrees to 1 degree over the first 0.01 seconds

Once this is done the circular block (with the plus and minus signs) which is the summing block calculates the error between the output from the revolute joints and the disturbance value and sends this to the PID control block which will try to reduce the error in several iterations depending on the gain values of the controller.

As per the scope block there will be a few oscillations before the bot is balanced perfectly at a desired angle (which in this case is taken to be around 1 degree) . The settling time is just under 3 seconds as per the graph.

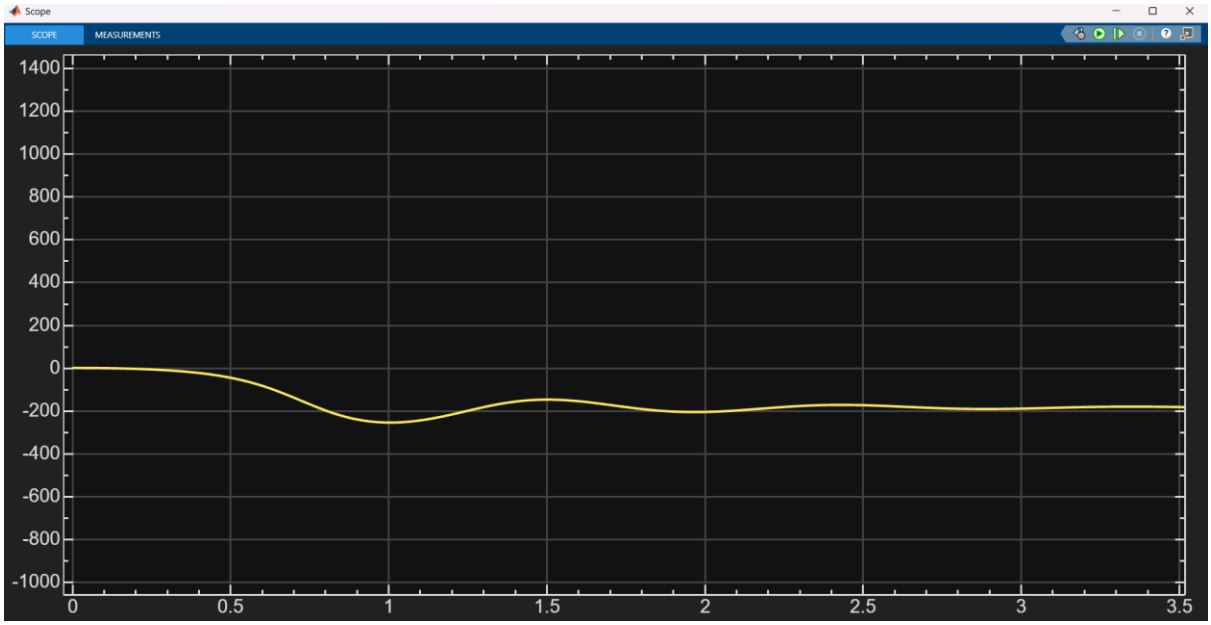


Fig 5. Oscillations in the system as depicted by the scope block

The PID controller has three turning parameters, Kp, Ki and Kd which are used to balance the system in position. These are the proportional, integral and derivative constants respectively.

The proportional constant-

a) Responds to the current error
b) Gives a stronger response if the error is more.
c) Causes the system to oscillate if its value is too high.

$$P(t) = K_p \cdot e(t)$$

The Integral constant-

a) Responds to accumulated past errors.
b) Eliminates steady state error.
c) May cause overshooting or instability if it's too high.

$$I(t) = K_i \cdot \int_0^t e(\tau)\, d\tau$$

The derivative constant-

a) Tries to control the rate at which the correction of error takes place.
b) Dampens the response in order to reduce the overshooting and oscillations.

$$D(t) = K_d \cdot \frac{de(t)}{dt}$$

Hence the final actuating signal is given by the following function:

$$u(t) = K_p e(t) + K_i \int e(t)\, dt + K_d \frac{de(t)}{dt}$$

Mechanics of the bot:

By deriving the equations involved in the mechanics of the bot we can obtain the expression for the transfer function of the system by considering torque as the input as angle theta as the output.

x(t) stands for the horizontal position of the wheel axis.

θ(t) stands for the tilt angle of the robot body from the vertical position.

We obtain the equations of the bot using the Euler lagrangian methods. Therefore we first need to find the potential and kinetic energies of the bot.

To find the kinetic energy we need the velocity components of the centre of mass.

- Horizontal velocity: $\dot{x}_{\text{CoM}} = \dot{x} + l\dot{\theta}\cos\theta$
- Vertical velocity: $\dot{y}_{\text{CoM}} = -l\dot{\theta}\sin\theta$

From this we obtain the total kinetic energy:

$$T = \frac{1}{2}m\left(\dot{x}_{\text{CoM}}^2 + \dot{y}_{\text{CoM}}^2\right) + \frac{1}{2}I\dot{\theta}^2$$

This can then be simplified to:

$$T = \frac{1}{2}m\dot{x}^2 + ml\dot{x}\dot{\theta}\cos\theta + \frac{1}{2}(I + ml^2)\dot{\theta}^2$$

The potential energy of the center of mass can be given by the following equation:

$$V = -mgl\cos\theta$$

Once we have the kinetic energy and the potential energy, we are ready to determine the Lagrangian term. Which is given by: L= T-V (difference between the kinetic and potential energies of the system)

$$\mathcal{L} = \frac{1}{2}m\dot{x}^2 + ml\dot{x}\dot{\theta}\cos\theta + \frac{1}{2}(I + ml^2)\dot{\theta}^2 + mgl\cos\theta$$

We then use the Euler Lagrangian equations to obtain two equations that give us the two crucial variables in this problem, they are the torque required(input) and the angle (output). Once we obtain these equations, we can formulate a transfer function with torque as input and angle as the output value of our system.

Euler Lagrangian equation:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}}\right) - \frac{\partial \mathcal{L}}{\partial q} = Q_q$$

Where Qp is the net generalized conservative force due to the torque that is required for the bot to balance itself and q is an independent physical parameter.

To get 2 equations we take q as x and theta:

For x:

$$\frac{\partial \mathcal{L}}{\partial \dot{x}} = m\dot{x} + ml\dot{\theta}\cos\theta$$

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{x}}\right) = m\ddot{x} + ml\ddot{\theta}\cos\theta - ml\dot{\theta}^2\sin\theta$$

$$\frac{\partial \mathcal{L}}{\partial x} = 0$$

$$\boxed{m\ddot{x} + ml\ddot{\theta}\cos\theta - ml\dot{\theta}^2\sin\theta = u}$$

For theta(angle):

$$\frac{\partial \mathcal{L}}{\partial \dot{\theta}} = ml\dot{x}\cos\theta + (I + ml^2)\dot{\theta}$$

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{\theta}}\right) = ml\ddot{x}\cos\theta - ml\dot{x}\dot{\theta}\sin\theta + (I + ml^2)\ddot{\theta}$$

$$\frac{\partial \mathcal{L}}{\partial \theta} = -ml\dot{x}\dot{\theta}\sin\theta - mgl\sin\theta$$

$$\boxed{(I + ml^2)\ddot{\theta} + ml\ddot{x}\cos\theta = mgl\sin\theta}$$

So, we finally get the equations of motion as:

$$\boxed{m\ddot{x} + ml\ddot{\theta}\cos\theta - ml\dot{\theta}^2\sin\theta = u}$$

$$\boxed{(I + ml^2)\ddot{\theta} + ml\ddot{x}\cos\theta = mgl\sin\theta}$$

And upon linearization about the upright steady position which is theta=0 degrees we get:

$$\boxed{m\ddot{x} + ml\ddot{\theta} = u}$$

$$\boxed{(I + ml^2)\ddot{\theta} + ml\ddot{x} = mgl\theta}$$

Which happens for very small values of theta.

The force u is basically the net horizontal force that is experienced due to the torque from the motor that tries to balance the bot in an upright position. It is basically equal to the reaction force exerted by the ground when the N-20 motor applies a torque on the wheel.

Next to design the control system we have to design the plant. In order to represent the plant, we can use the transfer function block that will take the force u(t) as the input and will provide theta(t) as the output.

The input force would be obtained from the PID controller that produces the actuating signal on the basis of the error signal between desired angle and actual angle.

We can then tune the PID parameters accordingly to get the desired output.

Conclusion:

The self-balancing bot project has successfully demonstrated the integration of Mechanical design and control theory that has been used to achieve dynamic stability. As a result, we have currently obtained a settling time of 3 seconds. By implementing the above Lagrangian-Eulerian equations of mechanics in MATLAB with the help of transfer functions we can further improve the bot's stability. The system's behavior was analyzed under the application of an external disturbance and a PID controller was used to reduce the tilt angle. Further work is being done currently on this project as it is an ongoing project. Alternative control methods like LQR (linear quadratic regulator) are being explored to ease tuning and to gain smoother control of the system. On the whole this project has so far reinforced key concepts in control theory, real-time feedback systems, and robotics laying a strong foundation for future work in mobile robotic systems and autonomous platforms.